



in collaboration with



C Programming Basics

Lab Report and Knowledge Check

Aaisma Wagle

Bsc (Hons) Computer Science with Artificial Intelligence

Softwarica College of IT and E-commerce

ST5003CMD Operating Systems, Security, and Networks

Rupak Rajbanshi

19th September, 2026

TABLE OF CONTENT

INTRODUCTION	1
GCC INSTALLATION PROCESS	1
Installing GCC on Linux	1
PROGRAM STRUCTURE AND EXAMPLES WITH RETURN VALUES AND EXIT STATUS	2
A. Basic Hello World Program	2
B. Program with User Input	3
C. Program with Formatted Output	5
GCC COMPILATION PROCESS	7
Four Stages of Compilation	8
Compilation Stage Examples	8
Stage 1: Preprocessing	8
Stage 2: Compilation to Assembly	9
Stage 3: Assembly to Object File	9
Stage 4: Linking	10
CHECKING EXIT STATUS	11
KNOWLEDGE CHECK QUESTIONS	12
Question 1	12
Question 2	12
Question 3	12
Question 4	13
Question 5	13
Question 6	13
LAB TASKS COMPLETED	14
CONCLUSION	15
GITHUB REPOSITORY	15
APPENDIX:	16
Stages of Compilation of userinfo.c	16
Stages of Compilation of userinput.c	18

LIST OF FIGURES

Figure 01: Make Lab2 Folder to Work With	1
Figure 02: apt update and gcc Version Check	1
Figure 03: Make helloworld.c File	2
Figure 04: helloworld.c File in Nano Editor	2
Figure 05: helloworld.c File View using cat Command	2
Figure 06: Run GCC compilation with return 0	3
Figure 07: Return Value Change in helloworld.c File	3
Figure 08: Run GCC compilation with return 1	3
Figure 09: Move Files to task1	3
Figure 10: Make task2 Repository and New userinput.c File	3
Figure 11: userinput.c File in Nano Editor	4
Figure 12: userinput.c File View using cat Command	4
Figure 13: Run GCC compiler with return 0	4
Figure 14: return Value Change inputuser.c File	4
Figure 15: Run GCC compiler with return 1	4
Figure 16: Make task2 Repository and New userinfo.c File	5
Figure 17: userinfo.c File in Nano Editor	5
Figure 18: userinfo.c File View using cat Command	5
Figure 19: Run GCC compiler with return 0	6
Figure 20: Return Value Change askinfo.c File	6
Figure 21: Run GCC compiler with return 1	6
Figure 22: Copy main task's C file inside task4 Repository	7
Figure 23: Runs and Displays preprocessed file	8
Figure 24: The Expanded Code and Headers included.	8
Figure 25: Translate to Assembly Language and Display it	9
Figure 26: Translate to Machine Code and Confirms if its machine code	9
Figure 27: Translate to Runnable File, Runs it and Shows Exit Status	10
Figure 28: Detailed Information of each Files	10
Figure 29: Make New task5 Folder and New return.c File	11
Figure 30: return.c File in Nano Editor	11

Figure 31: Run GCC compiler and Check Exit Status	11
Figure 32: Program that Declares, Asks User and Displays Entered Value back to User	13
Figure 33: Change Return Value to 50	14
Figure 34: echo Command Output	14
Figure 35: Compile userinfo.i and Display	15
Figure 36: Compile userinfo.s and Display	17
Figure 37: Compile userinfo.o and Confirms if its machine code	17
Figure 38: Compile to userinfo, Runs it, Shows Exit Status and Detailed Information of each files in userinfo	17
Figure 39: Compile userinput.i and Display	18
Figure 40: Compile userinput.s and Display	18
Figure 41: Compile userinput.o and Confirms if its machine code	18
Figure 42: Compile to userinput, Runs it, Shows Exit Status and Detailed Information of each files in userinput	18

INTRODUCTION

This is the lab report of Operating Systems, Security and Networks where the knowledge of C program structure, GCC compilation and exit status is being tested. It is done since the OS is what loads programs, runs it and reads it back during exit code. This shows the boundary between code and the operating system.

Sudo mkdir <repository name> //For creating a new folder. Sudo is used to access files that regular user account doesn't have access to

ls //list all files in the directory

```
wagleayden@Ayden:~/os-labs$ ls
C_Compile  week1
wagleayden@Ayden:~/os-labs$ sudo mkdir lab2
[sudo: authenticate] Password:
wagleayden@Ayden:~/os-labs$ ls
C_Compile  lab2  week1
wagleayden@Ayden:~/os-labs$ cd lab2
wagleayden@Ayden:~/os-labs/lab2$
```

Figure 01: Make Lab2 Folder to Work With

GCC INSTALLATION PROCESS

Installing GCC on Linux

```
wagleayden@Ayden: ~/os-labs
wagleayden@Ayden:~/os-labs$ sudo apt update
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:2 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [477 kB]
Get:5 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [112 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.6 kB]
Get:8 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [173 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [54.5 kB]
Get:10 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [612 kB]
Get:11 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.4 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [147 kB]
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [97.8 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [294 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [93.7 kB]
Get:16 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [188 kB]
Get:17 http://archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Components [1056 B]
Fetched 2752 kB in 5s (555 kB/s)
17 packages can be upgraded. Run 'apt list --upgradable' to see them.
wagleayden@Ayden:~/os-labs$ sudo apt install build-essential -y
build-essential is already the newest version (12.12ubuntu2.26.04.2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 17
wagleayden@Ayden:~/os-labs$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Figure 02: apt update and gcc Version Check

PROGRAM STRUCTURE AND EXAMPLES WITH RETURN VALUES AND EXIT STATUS

A. Basic Hello World Program

Here, a new file called 'helloworld.c' was made. Then the code of printing hello world was written in the file using nano editor and it was compiled using GCC compiler. When the binary file called 'printhello' runs, we use the echo command to check the return value. The value was changed from 0 to 1 to display a non-zero exit status. This confirmed that the program can report failure as well as success.

Sudo touch <filename> //For creating a new file.

Sudo nano <filename> //To open the file in nano editor to write or edit

It is saved using **Ctrl + X, Y** and **Enter**

```
wagleayden@Ayden:~/os-labs/lab2$ sudo touch helloworld.c
[sudo: authenticate] Password:
wagleayden@Ayden:~/os-labs/lab2$ ls
helloworld.c
wagleayden@Ayden:~/os-labs/lab2$ sudo nano helloworld.c
wagleayden@Ayden:~/os-labs/lab2$
```

Figure 03: Make helloworld.c File

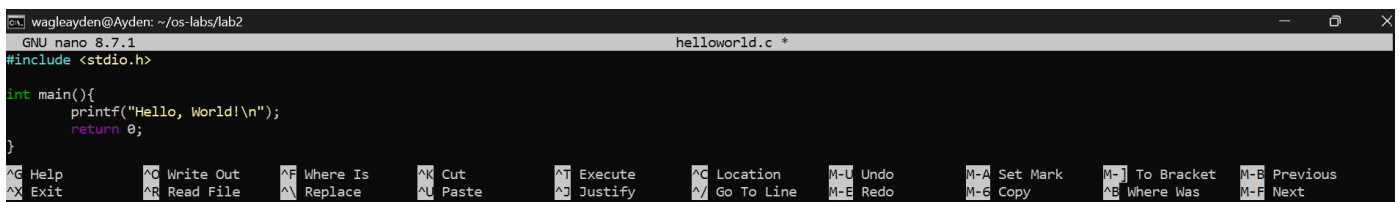


Figure 04: helloworld.c File in Nano Editor

cat <filename> //Prints a file's text directly to the terminal screen.



Figure 05: helloworld.c File View using cat Command

sudo gcc hello.c -o <executable binary file> //Compiles the source code into a runnable program

./<executable binary file> //Runs the binary file in the terminal

echo \$? //Prints the exit status code of the last executed file

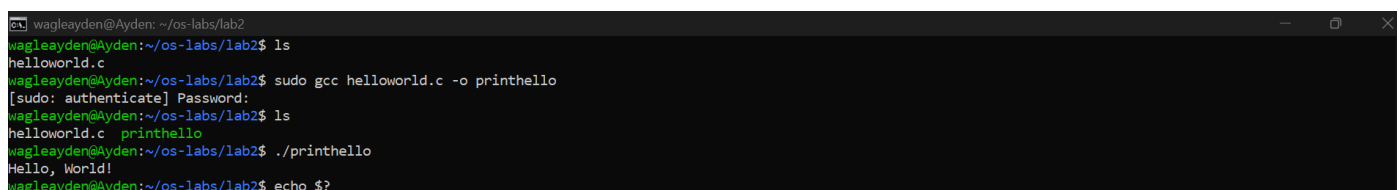


Figure 06: Run GCC compilation with return 0

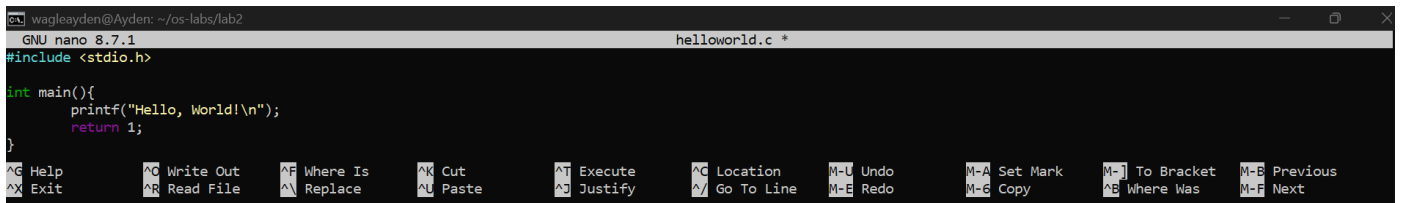


Figure 07: return Value Change in helloworld.c File



Figure 08: Run GCC compilation with return 1

B. Program with User Input

Before starting on the second task, a new folder for the previous task was made and the files were moved.

sudo mv <file name> <folder path> //moves the files to a certain folder.
cd <repository> //enters the folder
cd .. //Goes one step back to the repository the current repository it was in

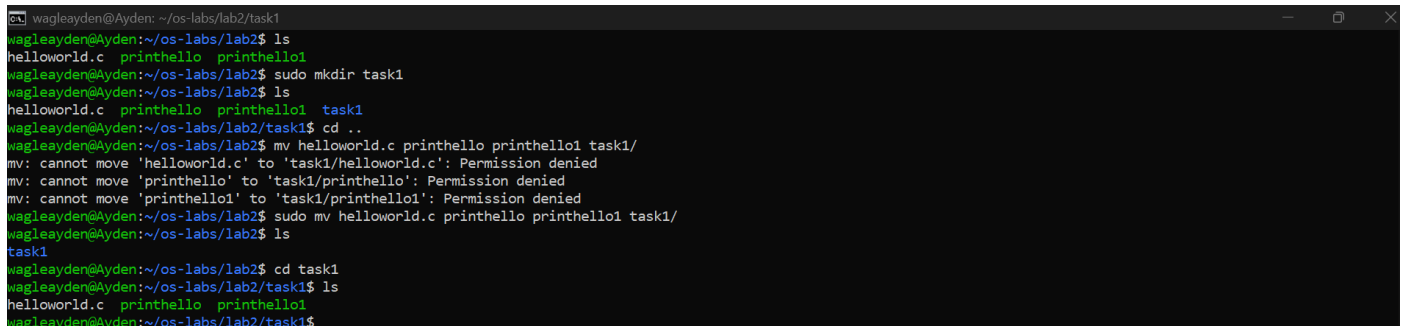


Figure 09: Move Files to task1

Here, a new file called 'userinput.c' was made. Then the code of asking the user for age and printing it was written in the file using nano editor and it was compiled using GCC compiler. When the binary file called 'askuserinput' runs, we use the echo command to check the return value. The value was changed from 0 to 1 to display a non-zero exit status. This confirmed that the program can report failure as well as success.

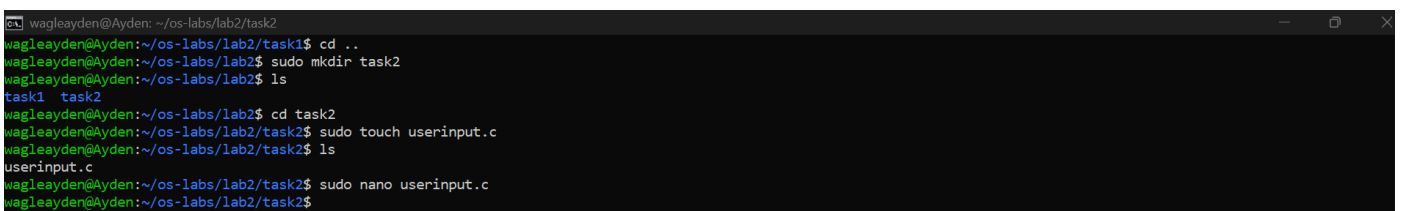
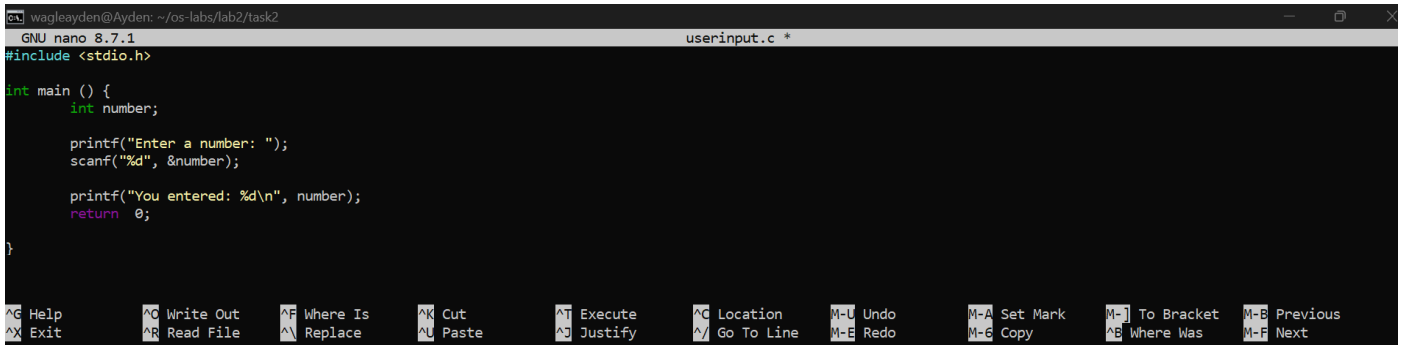


Figure 10: Make task2 Repository and New userinput.c File



```
GNU nano 8.7.1 userinput.c *
#include <stdio.h>

int main () {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 0;
}
```

Figure 11: userinput.c File in Nano Editor



```
wagleayden@Ayden: ~/os-labs/lab2/task2
wagleayden@Ayden:~/os-labs/lab2/task2$ cat userinput.c
#include <stdio.h>

int main () {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

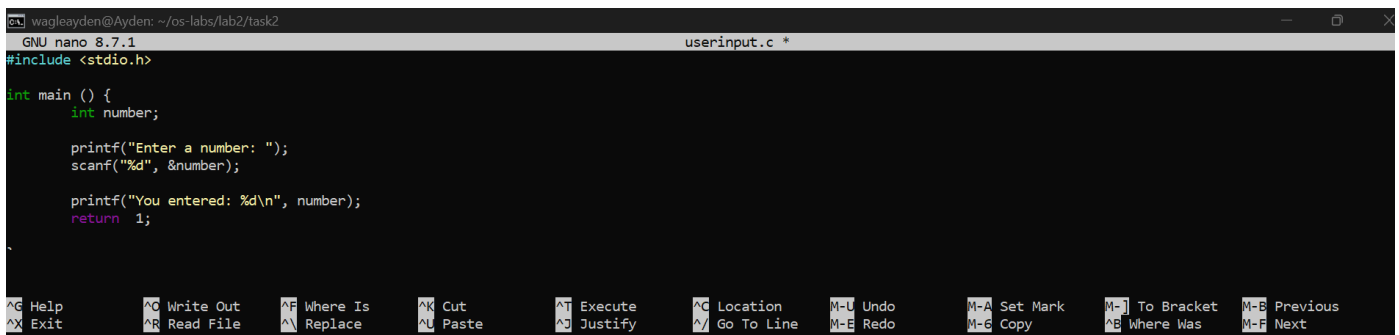
    printf("You entered: %d\n", number);
    return 0;
}
wagleayden@Ayden:~/os-labs/lab2/task2$
```

Figure 12: userinput.c File View using cat Command



```
wagleayden@Ayden: ~/os-labs/lab2/task2
wagleayden@Ayden:~/os-labs/lab2/task2$ sudo gcc userinput.c -o askuserinput
wagleayden@Ayden:~/os-labs/lab2/task2$ ./askuserinput
Enter a number: 30
You entered: 30
wagleayden@Ayden:~/os-labs/lab2/task2$ echo $?
0
wagleayden@Ayden:~/os-labs/lab2/task2$
```

Figure 13: Run GCC compiler with return 0



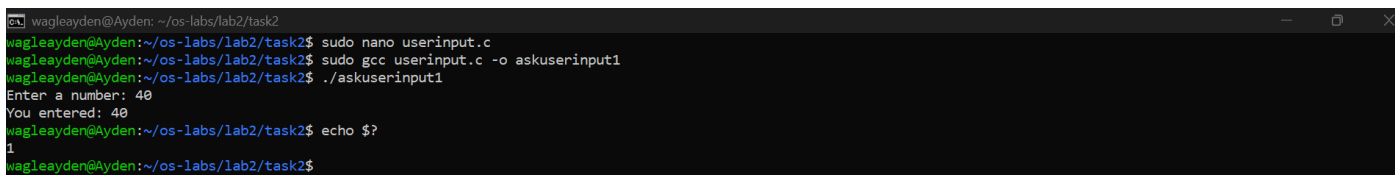
```
GNU nano 8.7.1 userinput.c *
#include <stdio.h>

int main () {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 1;
}
```

Figure 14: Return Value Change userinput.c File



```
wagleayden@Ayden: ~/os-labs/lab2/task2
wagleayden@Ayden:~/os-labs/lab2/task2$ sudo nano userinput.c
wagleayden@Ayden:~/os-labs/lab2/task2$ sudo gcc userinput.c -o askuserinput1
wagleayden@Ayden:~/os-labs/lab2/task2$ ./askuserinput1
Enter a number: 40
You entered: 40
wagleayden@Ayden:~/os-labs/lab2/task2$ echo $?
1
wagleayden@Ayden:~/os-labs/lab2/task2$
```

Figure 15: Run GCC compiler with return 1

C. Program with Formatted Output

Here, a new file called 'userinfo.c' was made. Then the code of asking the user for info and printing it was written in the file using nano editor and it was compiled using GCC compiler. When the binary file called 'askuserinfo' runs, we use the echo command to check the return value. The value was changed from 0 to 1 to display a non-zero exit status. This confirmed that the program can report failure as well as success.

```
wagleayden@Ayden: ~/os-labs/lab2/task2
wagleayden@Ayden:~/os-labs/lab2/task1$ cd ..
wagleayden@Ayden:~/os-labs/lab2$ sudo mkdir task2
wagleayden@Ayden:~/os-labs/lab2$ ls
task1 task2
wagleayden@Ayden:~/os-labs/lab2$ cd task2
wagleayden@Ayden:~/os-labs/lab2/task2$ sudo touch userinfo.c
wagleayden@Ayden:~/os-labs/lab2/task2$ ls
userinfo.c
wagleayden@Ayden:~/os-labs/lab2/task2$ sudo nano userinfo.c
wagleayden@Ayden:~/os-labs/lab2/task2$
```

Figure 16: Make task2 Repository and New userinfo.c File

```
GNU nano 8.7.1 userinfo.c *
#include <stdio.h>

int main() {
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m): ");
    scanf("%f", &height);

    printf("\n ---. User Info ---\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m\n", height);

    return 0;
}
```

Figure 17: userinfo.c File in Nano Editor

```
wagleayden@Ayden: ~/os-labs/lab2/task3
wagleayden@Ayden:~/os-labs/lab2/task3$ cat userinfo.c
#include <stdio.h>

int main() {
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m): ");
    scanf("%f", &height);

    printf("\n ---. User Info ---\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m\n", height);

    return 0;
}
```

Figure 18: userinfo.c File View using cat Command

```
wagleayden@Ayden: ~/os-labs/lab2/task3
wagleayden@Ayden:~/os-labs/lab2/task3$ sudo nano userinfo.c
wagleayden@Ayden:~/os-labs/lab2/task3$ sudo gcc userinfo.c -o askuserinfo
wagleayden@Ayden:~/os-labs/lab2/task3$ ./askuserinfo
Enter your name: Ayden
Enter your age: 13
Enter your height (m): 146

---. User Info ---
Name: Ayden
Age: 13 years
Height: 146.00 m
wagleayden@Ayden:~/os-labs/lab2/task3$ echo $?
0
wagleayden@Ayden:~/os-labs/lab2/task3$
```

Figure 19: Run GCC compiler with return 0

```
GNU nano 8.7.1 userinfo.c *
#include <stdio.h>

int main() {

    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m): ");
    scanf("%f", &height);

    printf("\n ---. User Info ---\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m\n", height);

    return 1;
}
```

Figure 20: Return Value Change askinfo.c File

```
wagleayden@Ayden: ~/os-labs/lab2/task3
wagleayden@Ayden:~/os-labs/lab2/task3$ sudo gcc userinfo.c -o askuserinfo1
wagleayden@Ayden:~/os-labs/lab2/task3$ ./askuserinfo1
Enter your name: Evgenia
Enter your age: 17
Enter your height (m): 156

---. User Info ---
Name: Evgenia
Age: 17 years
Height: 156.00 m
wagleayden@Ayden:~/os-labs/lab2/task3$ echo $?
1
wagleayden@Ayden:~/os-labs/lab2/task3$
```

Figure 21: Run GCC compiler with return 1

GCC COMPILATION PROCESS

task4 is made to hold the three codes for the three tasks done earlier. The main C file of each task is copied to start four compilation stages separately.

sudo cp <current file path> <new copied file path> //copies the files to necessary folder

. is used to go a step back in path.

```
wagleayden@Ayden: ~/os-labs/lab2/task4/userinfo
wagleayden@Ayden:~/os-labs/lab2/task4$ ls
wagleayden@Ayden:~/os-labs/lab2/task4$ sudo mkdir helloworld
wagleayden@Ayden:~/os-labs/lab2/task4$ sudo mkdir userinput
wagleayden@Ayden:~/os-labs/lab2/task4$ sudo mkdir userinfo
wagleayden@Ayden:~/os-labs/lab2/task4$ ls
helloworld  userinfo  userinput
wagleayden@Ayden:~/os-labs/lab2/task4$ sudo cp ../task1/helloworld.c ./helloworld/
wagleayden@Ayden:~/os-labs/lab2/task4$ sudo cp ../task2/userinput.c ./userinput/
wagleayden@Ayden:~/os-labs/lab2/task4$ sudo cp ../task3/userinfo.c ./userinfo/
wagleayden@Ayden:~/os-labs/lab2/task4$ ls
helloworld  userinfo  userinput
wagleayden@Ayden:~/os-labs/lab2/task4$ cd helloworld
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ ls
helloworld.c
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ cat helloworld.c
#include <stdio.h>

int main(){
    printf("Hello, World!\n");
    return 1;
}

wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ cd ..
wagleayden@Ayden:~/os-labs/lab2/task4$ cd userinput
wagleayden@Ayden:~/os-labs/lab2/task4/userinput$ ls
userinput.c
wagleayden@Ayden:~/os-labs/lab2/task4/userinput$ cat userinput.c
#include <stdio.h>

int main () {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 1;
}

wagleayden@Ayden:~/os-labs/lab2/task4/userinput$ cd ..
wagleayden@Ayden:~/os-labs/lab2/task4$ cd userinfo
wagleayden@Ayden:~/os-labs/lab2/task4/userinfo$ ls
userinfo.c
wagleayden@Ayden:~/os-labs/lab2/task4/userinfo$ cat userinfo.c
#include <stdio.h>

int main() {

    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m): ");
    scanf("%f", &height);

    printf("\n --- . User Info . ---\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m\n", height);

    return 1;
}

wagleayden@Ayden:~/os-labs/lab2/task4/userinfo$
```

Figure 22: Copy main task's C file inside task4 Repository

B. Four Stages of Compilation

During pre-processing in C, the compiler's preprocessor runs before actual compilation. It expands and replaces `#include <stdio.h>` with full contents of the header file. Then, the compiler translates the preprocessed C code into assembly language. The output is an assembly file that is human-readable.

The assembler converts assembly instructions into machine code i.e binary. The output is an object file, which contains raw machine code but is not yet executable. The linker combines one or more object files with required libraries. It resolves function calls like `printf` and produces the final executable. It creates an executable file.

C. Compilation Stage Examples

This is the example of task1. The full compilation of task2 and task3 will be in the appendix.

Stage 1: Preprocessing

gcc -E <C file (.c)> -o <preprocessed file (.i)> //Runs only the preprocessor stage.

Eg: gcc -E helloworld.c -o helloworld.i

cat <preprocessed file (.i)> | less //Displays the contents of the preprocessed file

Eg: cat helloworld.i | less

```
wagleayden@Ayden: ~/os-labs/lab2/task4/helloworld
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ ls
helloworld.c
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ sudo gcc -E helloworld.c -o helloworld.i
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ ls
helloworld.c  helloworld.i
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ cat helloworld.i | less
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$
```

Figure 23: Runs and Displays preprocessed file

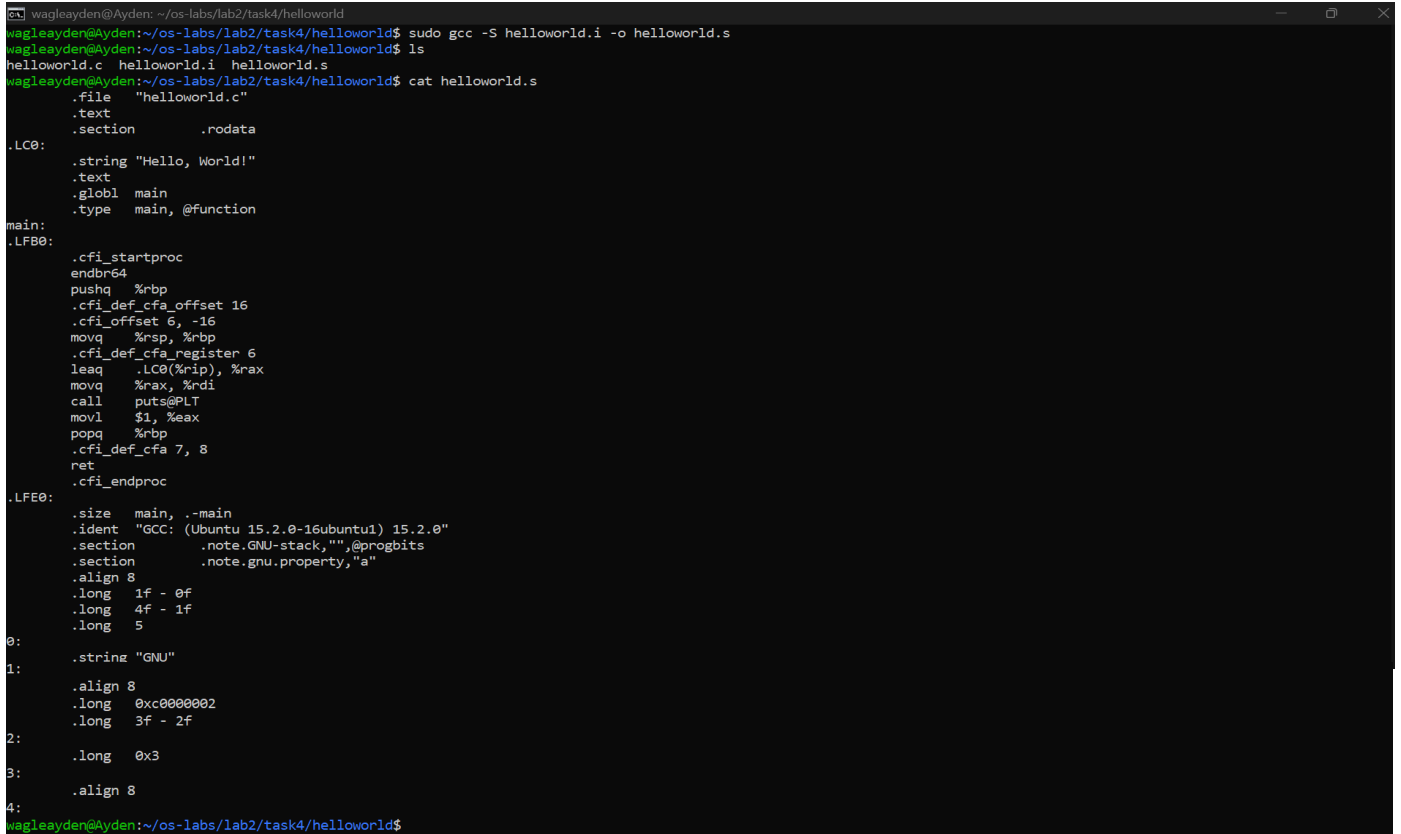
```
wagleayden@Ayden: ~/os-labs/lab2/task4/helloworld
# 0 "helloworld.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "helloworld.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 431 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 432 "/usr/include/features.h" 2 3 4
# 539 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 540 "/usr/include/features.h" 2 3 4
# 563 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 1 3 4
# 10 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs-64.h" 1 3 4
# 11 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 2 3 4
# 564 "/usr/include/features.h" 2 3 4
# 34 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 2 3 4
# 29 "/usr/include/stdio.h" 2 3 4
```

Figure 24: The Expanded Code and Headers included.

Stage 2: Compilation to Assembly

gcc -S <preprocessed file (.i)> -o <assembly code file (.s)> //Runs the compiler stage that translates C code into assembly language
Eg: gcc -S helloworld.i -o helloworld.s

cat <assembly code file (.s)> //Displays the contents of the assembly file.
Eg: cat helloworld.s



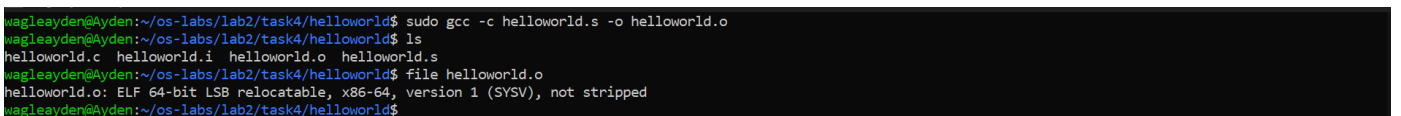
```
wagleayden@Ayden: ~/os-labs/lab2/task4/helloworld
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ sudo gcc -S helloworld.i -o helloworld.s
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ ls
helloworld.c  helloworld.i  helloworld.s
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ cat helloworld.s
.file "helloworld.c"
.text
.section .rodata
.LC0:
.string "Hello, World!"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
endbr64
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
leaq .LC0(%rip), %rax
movq %rax, %rdi
call puts@PLT
movl $1, %eax
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size main, .-main
.ident "GCC: (Ubuntu 15.2.0-16ubuntu1) 15.2.0"
.section .note.GNU-stack,"",@progbits
.section .note.gnu.property,"a"
.align 8
.long 1f - 0f
.long 4f - 1f
.long 5
0:
.string "GNU"
1:
.align 8
.long 0xc0000002
.long 3f - 2f
2:
.long 0x3
3:
.align 8
4:
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$
```

Figure 25: Translate to Assembly Language and Display it

Stage 3: Assembly to Object File

gcc -c <assembly code file (.s)> -o <object file (.o)> //Runs the assembler stage and converts the human-readable assembly instructions into machine code.
Eg: gcc -c helloworld.s -o helloworld.o

file <object file (.o)> //Checks the type of the object file and confirms it is machine code
Eg: file helloworld.o



```
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ sudo gcc -c helloworld.s -o helloworld.o
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ ls
helloworld.c  helloworld.i  helloworld.o  helloworld.s
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ file helloworld.o
helloworld.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$
```

Figure 26: Translate to Machine Code and Confirms if its machine code

Stage 4: Linking

gcc <object file (.o)> -o <executable file> //Runs the linker stage, combines the object file with needed

Eg: gcc helloworld.s -o hello libraries and resolves functions to produce the final runnable program..

./<executable file> //Runs the program.

Eg: ./hello

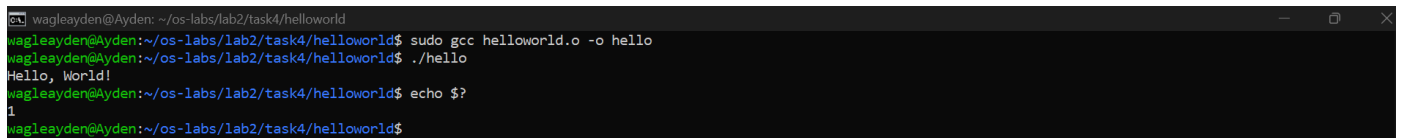
echo \$? //Shows the exit status of the last program run. 0 means success

ls -lh helloworld.c helloworld.i helloworld.s helloworld.o hello //lists all five files with detailed info

Compare how each stage of compilation changes file size.

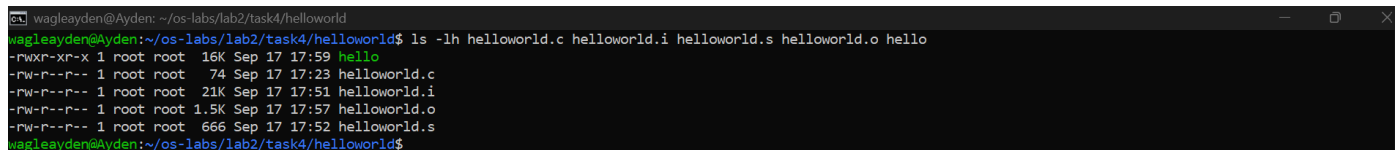
-l : long listing format that shows permissions, owner, size, date, etc.

-h : human-readable sizes in KB or MB instead of raw bytes



```
wagleayden@Ayden: ~/os-labs/lab2/task4/helloworld
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ sudo gcc helloworld.o -o hello
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ ./hello
Hello, World!
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ echo $?
1
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$
```

Figure 27: Translate to Runnable File, Runs it and Shows Exit Status



```
wagleayden@Ayden: ~/os-labs/lab2/task4/helloworld
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$ ls -lh helloworld.c helloworld.i helloworld.s helloworld.o hello
-rwxr-xr-x 1 root root 16K Sep 17 17:59 hello
-rw-r--r-- 1 root root 74 Sep 17 17:23 helloworld.c
-rw-r--r-- 1 root root 21K Sep 17 17:51 helloworld.i
-rw-r--r-- 1 root root 1.5K Sep 17 17:57 helloworld.o
-rw-r--r-- 1 root root 666 Sep 17 17:52 helloworld.s
wagleayden@Ayden:~/os-labs/lab2/task4/helloworld$
```

Figure 28: Detailed Information of each Files

CHECKING EXIT STATUS

Echo \$? Command is used to check the return value. This command prints the exit status of the last program that was run. In Linux, an exit status of 0 means the program finished successfully. If the program returns a different value then `echo \$?` will display that number instead.

```
wagleayden@Ayden: ~/os-labs/lab2/task5
wagleayden@Ayden:~/os-labs/lab2$ ls
task1 task2 task3 task4
wagleayden@Ayden:~/os-labs/lab2$ sudo mkdir task5
wagleayden@Ayden:~/os-labs/lab2$ ls
task1 task2 task3 task4 task5
wagleayden@Ayden:~/os-labs/lab2$ cd task5
wagleayden@Ayden:~/os-labs/lab2/task5$
wagleayden@Ayden:~/os-labs/lab2/task5$ sudo touch return.c
wagleayden@Ayden:~/os-labs/lab2/task5$ sudo nano return.c
wagleayden@Ayden:~/os-labs/lab2/task5$
```

Figure 29: Make New task5 Folder and New return.c File

```
GNU nano 8.7.1 return.c *
#include <stdio.h>
.
.
.
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^-A Set Mark   ^-] To Bracket ^-B Previous
^X Exit      ^R Read File  ^_ Replace    ^V Paste      ^J Justify    ^/_ Go To Line   ^-E Redo      ^-G Copy      ^_- Where Was  ^-F Next
```

Figure 30 return.c File in Nano Editor

```
wagleayden@Ayden: ~/os-labs/lab2/task5
wagleayden@Ayden:~/os-labs/lab2/task5$ sudo gcc return.c -o return
wagleayden@Ayden:~/os-labs/lab2/task5$ ./return
Program executed successfully
wagleayden@Ayden:~/os-labs/lab2/task5$ ls
return return.c
wagleayden@Ayden:~/os-labs/lab2/task5$ echo $?
0
wagleayden@Ayden:~/os-labs/lab2/task5$
```

Figure 31: Run GCC compiler and Check Exit Status

KNOWLEDGE CHECK QUESTIONS

Question 1

What is the purpose of the *#include* directive in C programming? Provide an example of its usage.

The main purpose of *#include* is to import external files into the program. It also helps you create your own header and include them in other programs instead of rewriting the functions.

Example:

```
#include <stdio.h>
```

This line includes the Standard Input Output header, which allows the program to use `printf()` and `scanf()`.

Question 2

Explain the difference between the four stages of compilation. What file extension is produced at each stage?

The compilation of a C program happens in four separate stages. Each stage produces a different type of output file:

1. **Preprocessing:** Expands *#include* headers and macros, and remove comments. It produces an expanded source code.

Code: `gcc -E <C file (.c)> -o <preprocessed file (.i)>`

2. **Compilation:** Translates the preprocessed C code into assembly language . It produces an assembly code.

Code: `gcc -S <preprocessed file (.i)> -o <assembly code file (.s)>`

3. **Assembling:** Converts the assembly code into machine code producing an object file. It produces an object code.

Code: `gcc -c <assembly code file (.s)> -o <object file (.o)>`

4. **Linking:** Combines the object file with any required libraries and other object files to produce a final, runnable program. It produces an executable file.

Code: `gcc <object file (.o)> -o <executable file>`

`./<executable file>`

Each stage takes the output of the previous one and moves it closer to a program the operating system can actually run.

Question 3

What does a return value of 0 from the main() function indicate to the operating system? What about a non-zero return value?

A return value of 0 from main() tells the operating system that the program executed successfully and finished without errors.

A non-zero return value like 1 tells the operating system that the program encountered a problem or ended abnormally. Different non-zero values can be used to represent different types of errors which is useful for scripts that check `echo $?`. It reacts differently depending on the exit code.

Question 4

Write a C program that: Declares an integer variable, Asks the user to input a value, Displays the entered value back to the user.

```
#include <stdio.h>

int main () {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
    return 1;
}
```

Figure 32: Program that Declares, Asks User and Displays Entered Value back to User

This program declares an integer variable, number. It asks the user to input a value using scanf(), and then displays the value back to the user using printf().

Question 5

What GCC command would you use to:

Only preprocess a file without compiling?

Code: gcc -E <C file (.c)> -o <preprocessed file (.i)>

Generate assembly code but not assemble it?

Code: gcc -S <preprocessed file (.i)> -o <assembly code file (.s)>

Create an object file without linking?

Code: gcc -c <assembly code file (.s)> -o <object file (.o)>

Question 6

Explain why C is called a "compiled language" and how this differs from interpreted languages.

C is called a compiled language because the entire source code is translated into machine code by a compiler before the program is run. This produces an executable file that the CPU can run directly without needing the original source code or the compiler present at run time.

This is different from an interpreted language where the source code is read and executed line by line by an interpreter each time the program runs. Interpreted programs are generally slower because translation happens at run time. However, they are often easier to test and debug since there is no separate compilation step. Compiled programs like C are faster to run since all translation work is done in advance.

LAB TASKS COMPLETED

Task 1: Compiled hello.c with return 0 and verified exit status using echo \$?

Task 2: Modified program to return 1 and observed different exit status

Task 3: Created program accepting user input and displaying it

Task 4: Executed all four compilation stages separately and documented output files

Task 5: Compiled and ran program returning integer value 50

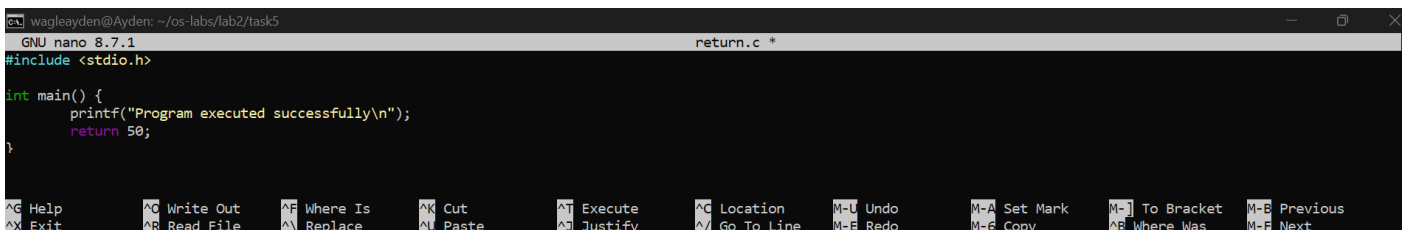
Task 1 and **Task 2** have been done in [Basic Hello World Program](#).

Task 3 is the *userinput.c* which has been added inside [Program with User Input](#)

Task 4 is the four compilation stages.

- helloworld.c: [Stages of Compilation of userinfo.c](#)
- userinfo.c: [Stages of Compilation of userinfo.c](#)
- userinput.c: [Stages of Compilation of userinput.c](#)

Task 5:

A screenshot of a terminal window showing the nano text editor. The editor is editing a file named 'return.c'. The code visible is:

```
#include <stdio.h>

int main() {
    printf("Program executed successfully\n");
    return 50;
}
```

 The bottom status bar of the nano editor shows various keyboard shortcuts like ^G Help, ^O Write Out, ^F Where Is, etc.

Figure 33: Change Return Value to 50

A screenshot of a terminal window showing the execution of several commands. The output is as follows:

```
wagleayden@Ayden: ~/os-labs/lab2/task5
wagleayden@Ayden:~/os-labs/lab2/task5$ ls
return.c
wagleayden@Ayden:~/os-labs/lab2/task5$ sudo nano return.c
wagleayden@Ayden:~/os-labs/lab2/task5$ sudo gcc return.c -o return50
wagleayden@Ayden:~/os-labs/lab2/task5$ ls
return.c  return50
wagleayden@Ayden:~/os-labs/lab2/task5$ ./return50
Program executed successfully
wagleayden@Ayden:~/os-labs/lab2/task5$ echo $?
50
wagleayden@Ayden:~/os-labs/lab2/task5$
```

Figure 34: echo Command Output

CONCLUSION

This lab provides practical experience with the fundamentals of C as shown in the linux system as shown above. The code is also run and written. We have also separately checked the role of preprocessor directives, the main() function, and the standard input and output functions. The four compilations stages were also carried out. The stages show how machine code is turned into executable files which the operating system can load and run. Testing different return values shows how a program communicates success or failure to the shell. A return value of 0 means it is successful and non-zero values like 1 and 50 are spotted by echo \$?.

This confirms that the exit status is passed from the program to the operating system.

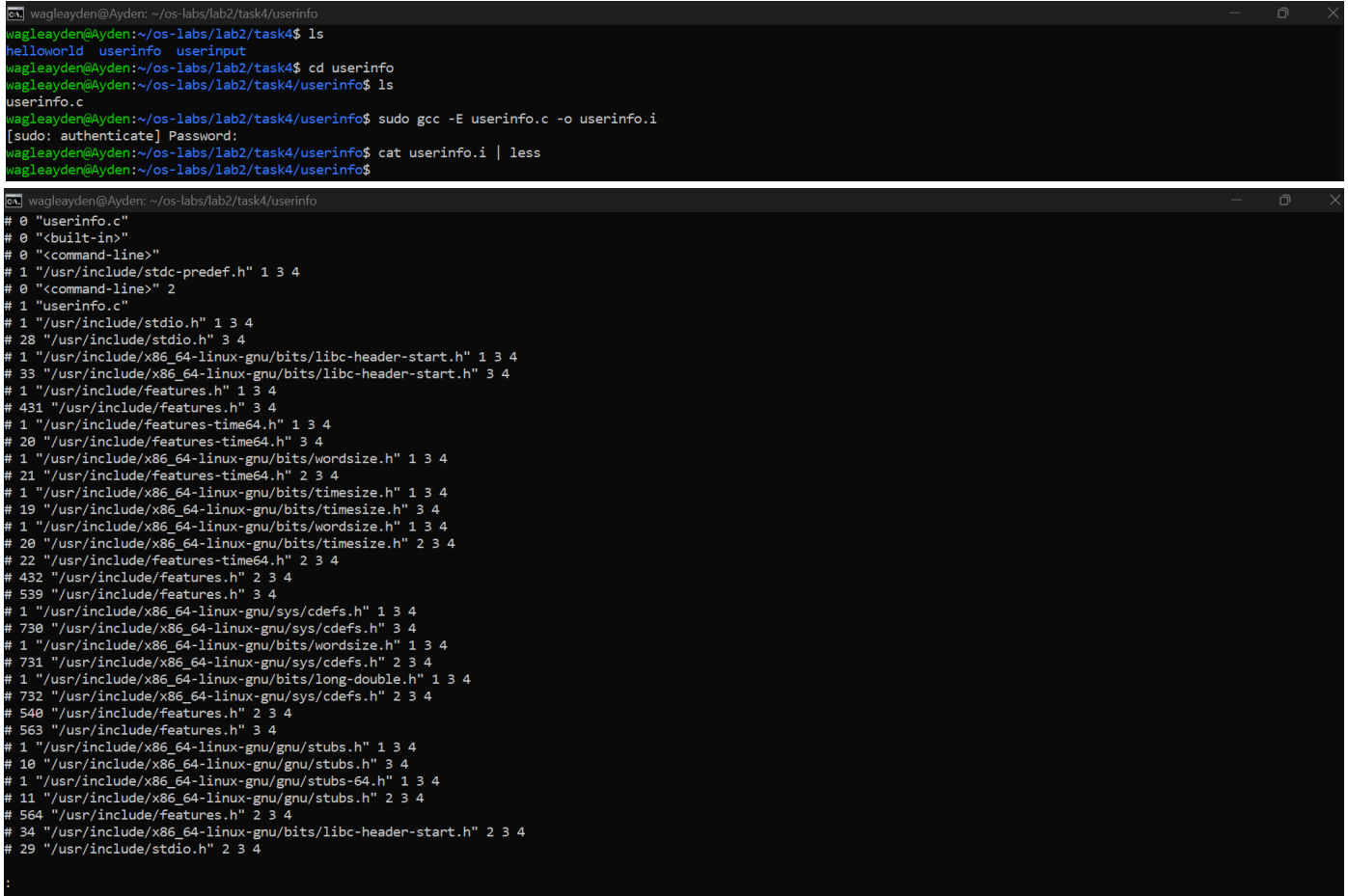
In conclusion, the lab objectives were met, that is to understand the compilation pipeline and exit codes. They are essential background for the systems programming, process management, and security topics covered later in this module.

GITHUB REPOSITORY

[GitHub Repository](#)

APPENDIX:

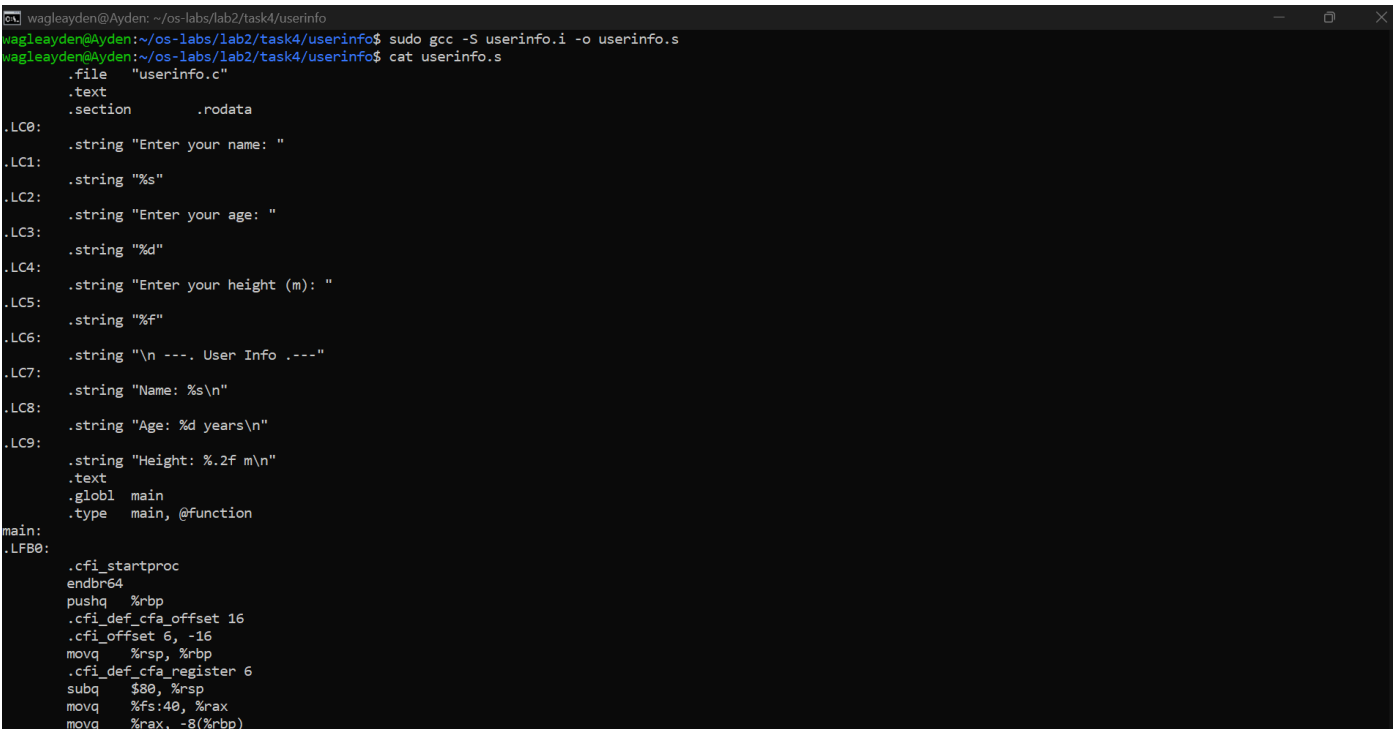
Stages of Compilation of userinfo.c



```
wagleayden@Ayden: ~/os-labs/lab2/task4/userinfo
wagleayden@Ayden:~/os-labs/lab2/task4$ ls
helloworld  userinfo  userinput
wagleayden@Ayden:~/os-labs/lab2/task4$ cd userinfo
wagleayden@Ayden:~/os-labs/lab2/task4/userinfo$ ls
userinfo.c
wagleayden@Ayden:~/os-labs/lab2/task4/userinfo$ sudo gcc -E userinfo.c -o userinfo.i
[sudo: authenticate] Password:
wagleayden@Ayden:~/os-labs/lab2/task4/userinfo$ cat userinfo.i | less
wagleayden@Ayden:~/os-labs/lab2/task4/userinfo$
```

```
# 0 "userinfo.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "userinfo.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 431 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 432 "/usr/include/features.h" 2 3 4
# 539 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 540 "/usr/include/features.h" 2 3 4
# 563 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 1 3 4
# 10 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs-64.h" 1 3 4
# 11 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 2 3 4
# 564 "/usr/include/features.h" 2 3 4
# 34 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 2 3 4
# 29 "/usr/include/stdio.h" 2 3 4
:
```

Figure 35: Compile userinfo.i and Display



```
wagleayden@Ayden: ~/os-labs/lab2/task4/userinfo
wagleayden@Ayden:~/os-labs/lab2/task4/userinfo$ sudo gcc -S userinfo.i -o userinfo.s
wagleayden@Ayden:~/os-labs/lab2/task4/userinfo$ cat userinfo.s
.file "userinfo.c"
.text
.section .rodata
.LC0:
.string "Enter your name: "
.LC1:
.string "%s"
.LC2:
.string "Enter your age: "
.LC3:
.string "%d"
.LC4:
.string "Enter your height (m): "
.LC5:
.string "%f"
.LC6:
.string "\n ---. User Info. ---"
.LC7:
.string "Name: %s\n"
.LC8:
.string "Age: %d years\n"
.LC9:
.string "Height: %.2f m\n"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
endbr64
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
subq $80, %rsp
movq %fs:40, %rax
movq %rax, -8(%rbp)
```

```
wagleayden@Ayden: ~/os-labs/lab2/task4/userinfo
xorl    %eax, %eax
leaq    .LC0(%rip), %rax
movq    %rax, %rdi
movl    $0, %eax
call    printf@PLT
leaq    -64(%rbp), %rax
leaq    .LC1(%rip), %rdx
movq    %rax, %rsi
movq    %rdx, %rdi
movl    $0, %eax
call    __isoc23_scanf@PLT
leaq    .LC2(%rip), %rax
movq    %rax, %rdi
movl    $0, %eax
call    printf@PLT
leaq    -72(%rbp), %rax
leaq    .LC3(%rip), %rdx
movq    %rax, %rsi
movq    %rdx, %rdi
movl    $0, %eax
call    __isoc23_scanf@PLT
leaq    .LC4(%rip), %rax
movq    %rax, %rdi
movl    $0, %eax
call    printf@PLT
leaq    -68(%rbp), %rax
leaq    .LC5(%rip), %rdx
movq    %rax, %rsi
movq    %rdx, %rdi
movl    $0, %eax
call    __isoc23_scanf@PLT
leaq    .LC6(%rip), %rax
movq    %rax, %rdi
call    puts@PLT
leaq    -64(%rbp), %rax
leaq    .LC7(%rip), %rdx
movq    %rax, %rsi
movq    %rdx, %rdi
movl    $0, %eax
call    printf@PLT

wagleayden@Ayden: ~/os-labs/lab2/task4/userinfo
xorl    %eax, %eax
leaq    .LC0(%rip), %rax
movq    %rax, %rdi
movl    $0, %eax
call    printf@PLT
leaq    -64(%rbp), %rax
leaq    .LC1(%rip), %rdx
movq    %rax, %rsi
movq    %rdx, %rdi
movl    $0, %eax
call    __isoc23_scanf@PLT
leaq    .LC2(%rip), %rax
movq    %rax, %rdi
movl    $0, %eax
call    printf@PLT
leaq    -72(%rbp), %rax
leaq    .LC3(%rip), %rdx
movq    %rax, %rsi
movq    %rdx, %rdi
movl    $0, %eax
call    __isoc23_scanf@PLT
leaq    .LC4(%rip), %rax
movq    %rax, %rdi
movl    $0, %eax
call    printf@PLT
leaq    -68(%rbp), %rax
leaq    .LC5(%rip), %rdx
movq    %rax, %rsi
movq    %rdx, %rdi
movl    $0, %eax
call    __isoc23_scanf@PLT
leaq    .LC6(%rip), %rax
movq    %rax, %rdi
call    puts@PLT
leaq    -64(%rbp), %rax
leaq    .LC7(%rip), %rdx
movq    %rax, %rsi
movq    %rdx, %rdi
movl    $0, %eax
call    printf@PLT

wagleayden@Ayden: ~/os-labs/lab2/task4/userinfo
call    printf@PLT
movl    -72(%rbp), %eax
leaq    .LC8(%rip), %rdx
movl    %eax, %esi
movq    %rdx, %rdi
movl    $0, %eax
call    printf@PLT
movss   -68(%rbp), %xmm0
pxor    %xmm1, %xmm1
cvtss2sd    %xmm0, %xmm1
movq    %xmm1, %rax
leaq    .LC9(%rip), %rdx
movq    %rax, %xmm0
movq    %rdx, %rdi
movl    $1, %eax
call    printf@PLT
movl    $1, %eax
movq    -8(%rbp), %rdx
subq    %fs:40, %rdx
je      .L3
call    __stack_chk_fail@PLT
.L3:
leave
.cfi_def_cfa 7, 8
ret
```

```

.cfi_endproc
.LFE0:
.size main, .-main
.ident "GCC: (Ubuntu 15.2.0-16ubuntu1) 15.2.0"
.section .note.GNU-stack,"",@progbits
.section .note.gnu.property,"a"
.align 8
.long 1f - 0f
.long 4f - 1f
.long 5
0:
.string "GNU"
1:
.align 8
.long 0xc0000002
2:
.long 3f - 2f
3:
.long 0x3
4:
.align 8

```

Figure 36: Compile userinfo.s and Display

```

wagleayden@Ayden:~/os-labs/lab2/task4/userinfo$ sudo gcc -c userinfo.s -o userinfo.o
wagleayden@Ayden:~/os-labs/lab2/task4/userinfo$ file userinfo.o
userinfo.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped

```

Figure 37: Compile userinfo.o and Confirms if its machine code

```

wagleayden@Ayden:~/os-labs/lab2/task4/userinfo$ sudo gcc userinfo.o -o userinfo
wagleayden@Ayden:~/os-labs/lab2/task4/userinfo$ ./userinfo
Enter your name: Ayden
Enter your age: 50
Enter your height (m): 168

---. User Info ---
Name: Ayden
Age: 50 years
Height: 168.00 m
wagleayden@Ayden:~/os-labs/lab2/task4/userinfo$ echo $?
0
wagleayden@Ayden:~/os-labs/lab2/task4/userinfo$ ls -lh userinfo.c userinfo.i userinfo.s userinfo.o userinfo
-rwxr-xr-x 1 root root 16K Sep 19 07:10 userinfo
-rw-r--r-- 1 root root 389 Sep 17 17:24 userinfo.c
-rw-r--r-- 1 root root 22K Sep 19 07:02 userinfo.i
-rw-r--r-- 1 root root 2.4K Sep 19 07:09 userinfo.o
-rw-r--r-- 1 root root 2.1K Sep 19 07:08 userinfo.s
wagleayden@Ayden:~/os-labs/lab2/task4/userinfo$

```

Figure 38: Compile to userinfo, Runs it, Shows Exit Status and Detailed Information of each files in userinfo

Stages of Compilation of userInput.c

```

wagleayden@Ayden: ~/os-labs/lab2/task4/userinput
wagleayden@Ayden:~/os-labs/lab2/task4$ ls
helloworld userinfo userInput
wagleayden@Ayden:~/os-labs/lab2/task4$ cd userInput
wagleayden@Ayden:~/os-labs/lab2/task4/userinput$ ls
userinput.c
wagleayden@Ayden:~/os-labs/lab2/task4/userinput$ sudo gcc -E userinput.c -o userinput.i
[sudo: authenticate] Password:
wagleayden@Ayden:~/os-labs/lab2/task4/userinput$ cat userinput.i | less
wagleayden@Ayden:~/os-labs/lab2/task4/userinput$

```

```

# 0 "userinput.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "userinput.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 431 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 432 "/usr/include/features.h" 2 3 4
# 539 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 540 "/usr/include/features.h" 2 3 4
# 563 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 1 3 4
# 10 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs-64.h" 1 3 4
# 11 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 2 3 4
# 564 "/usr/include/features.h" 2 3 4
# 34 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 2 3 4
# 29 "/usr/include/stdio.h" 2 3 4

```

Figure 39: Compile userinput.i and Display

```
wagleayden@Ayden: ~/os-labs/lab2/task4/userinput
wagleayden@Ayden:~/os-labs/lab2/task4/userinput$ sudo gcc -S userinput.i -o userinput.s
wagleayden@Ayden:~/os-labs/lab2/task4/userinput$ cat userinput.s
.file "userinput.c"
.text
.section .rodata
.LC0:
.string "Enter a number: "
.LC1:
.string "%d"
.LC2:
.string "You entered: %d\n"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
endbr64
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
subq $16, %rsp
movq %fs:40, %rax
movq %rax, -8(%rbp)
xorl %eax, %eax
leaq .LC0(%rip), %rax
movq %rax, %rdi
movl $0, %eax
call printf@PLT
leaq -12(%rbp), %rax
leaq .LC1(%rip), %rdx
movq %rax, %rsi
movq %rdx, %rdi
movl $0, %eax
call __isoc23_scanf@PLT
movl -12(%rbp), %eax
leaq .LC2(%rip), %rdx
movl %eax, %esi

movl $1, %eax
movq -8(%rbp), %rdx
subq %fs:40, %rdx
je .L3
call __stack_chk_fail@PLT
.L3:
leave
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size main, .-main
.ident "GCC: (Ubuntu 15.2.0-16ubuntu1) 15.2.0"
.section .note.GNU-stack,"",@progbits
.section .note.gnu.property,"a"
.align 8
.long 1f - 0f
.long 4f - 1f
.long 5
0:
.string "GNU"
1:
.align 8
.long 0xc0000002
.long 3f - 2f
2:
.long 0x3
3:
.align 8
4:
```

Figure 40: Compile userinput.s and Display

```
wagleayden@Ayden:~/os-labs/lab2/task4/userinput$ sudo gcc -c userinput.s -o userinput.o
wagleayden@Ayden:~/os-labs/lab2/task4/userinput$ file userinput.o
userinput.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
```

Figure 41: Compile userinput.o and Confirms if its machine code

```
wagleayden@Ayden:~/os-labs/lab2/task4/userinput$ sudo gcc userinput.o -o userinput
wagleayden@Ayden:~/os-labs/lab2/task4/userinput$ ./userinput
Enter a number: 4
You entered: 4
wagleayden@Ayden:~/os-labs/lab2/task4/userinput$ echo $?
1
wagleayden@Ayden:~/os-labs/lab2/task4/userinput$ ls -lh userinput.c userinput.i userinput.s userinput.o userinput
-rwxr-xr-x 1 root root 16K Sep 19 07:42 userinput
-rw-r--r-- 1 root root 155 Sep 17 17:23 userinput.c
-rw-r--r-- 1 root root 22K Sep 19 07:36 userinput.i
-rw-r--r-- 1 root root 1.8K Sep 19 07:41 userinput.o
-rw-r--r-- 1 root root 1.1K Sep 19 07:40 userinput.s
wagleayden@Ayden:~/os-labs/lab2/task4/userinput$
```

Figure 42: Compile to userinput, Runs it, Shows Exit Status and Detailed Information of each files in userinput